Amazon Product Analysis – EDA

Data Cleaning • Visualization • Insights • Recommendations

Objective

Analyze Amazon product data to understand pricing, discounts, ratings, category trends, and customer behavior.

Dataset Overview

Includes product name, category, actual price, discounted price, rating, reviews, and product details.

Data Cleaning

- Removed duplicates
- Converted price columns
- Cleaned discounts
- Created short categories
- Created review count

1-

```python
# Remove ₹ sign, commas, % etc.
df['discounted_price'] = df['discounted_price'].str.replace('₹', '').str.replace(',', '').astype(float)
df['actual_price'] = df['actual_price'].str.replace('₹', '').str.replace(',', '').astype(float)
df['discount_percentage'] = df['discount_percentage'].str.replace('%', '').astype(float)
df['rating'] = pd.to_numeric(df['rating'], errors='coerce')
df['rating_count'] = pd.to_numeric(df['rating_count'], errors='coerce')
```

2-

```python
df.isnull().sum()
```

```
product_id                0
product_name              0
category                  0
discounted_price          0
actual_price              0
discount_percentage       0
rating                    1
rating_count           1139
about_product             0
user_id                   0
user_name                 0
review_id                 0
review_title              0
review_content            0
img_link                  0
product_link              0
dtype: int64
```

```
!pip install matplotlib
```

3-

```
Requirement already satisfied: matplotlib in c:\users\shrad\miniconda3\lib\site-packages (3.10.7)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib) (4.61.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: numpy>=1.23 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib) (2.3.5)
Requirement already satisfied: packaging>=20.0 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib) (12.0.0)
Requirement already satisfied: pyparsing>=3 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\shrad\miniconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

```
!pip install seaborn
```

4-

```
Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\users\shrad\miniconda3\lib\site-packages (from seaborn) (2.3.5)
Requirement already satisfied: pandas>=1.2 in c:\users\shrad\miniconda3\lib\site-packages (from seaborn) (2.3.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\users\shrad\miniconda3\lib\site-packages (from seaborn) (3.10.7)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.61.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.2)
Requirement already satisfied: pillow>=8 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.0.0)
Requirement already satisfied: pyparsing>=3 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\shrad\miniconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post
0)
Requirement already satisfied: pytz>=2020.1 in c:\users\shrad\miniconda3\lib\site-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\shrad\miniconda3\lib\site-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\shrad\miniconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn)
(1.17.0)
Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)
```

```
Successfully installed seaborn-0.13.2
```

5- [11]:

```
import matplotlib.pyplot as plt
import seaborn as sns
```

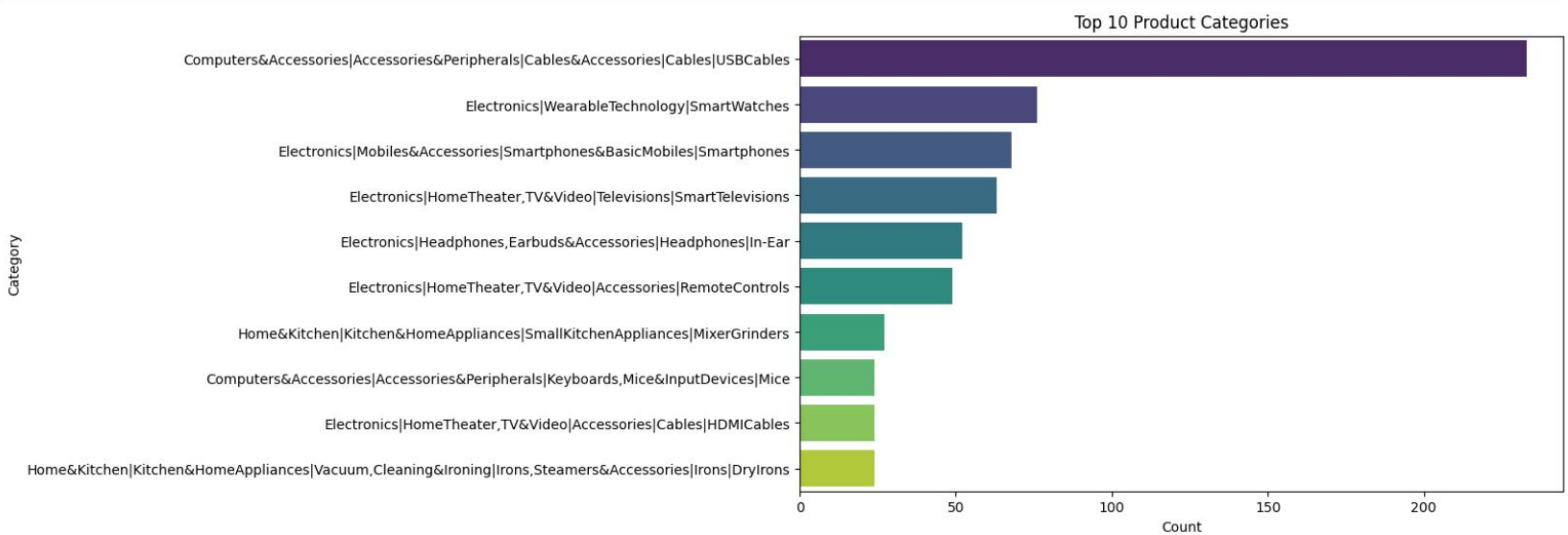
Exploratory Data Analysis

- Top 10 Products
- Top 10 Product Category
- Rating analysis
- Price vs Rating
- Top 10 Category highest Average price
- Top 10 Product With Highest Discount
- Top 10 Most Reviewed Product
- Top 10 High Rated Product
- Correlation heatmap
- Actual Price VS Discount Price Distribution

```
df['short_category'] = df['category'].apply(lambda x: x.split('|')[-1])
```

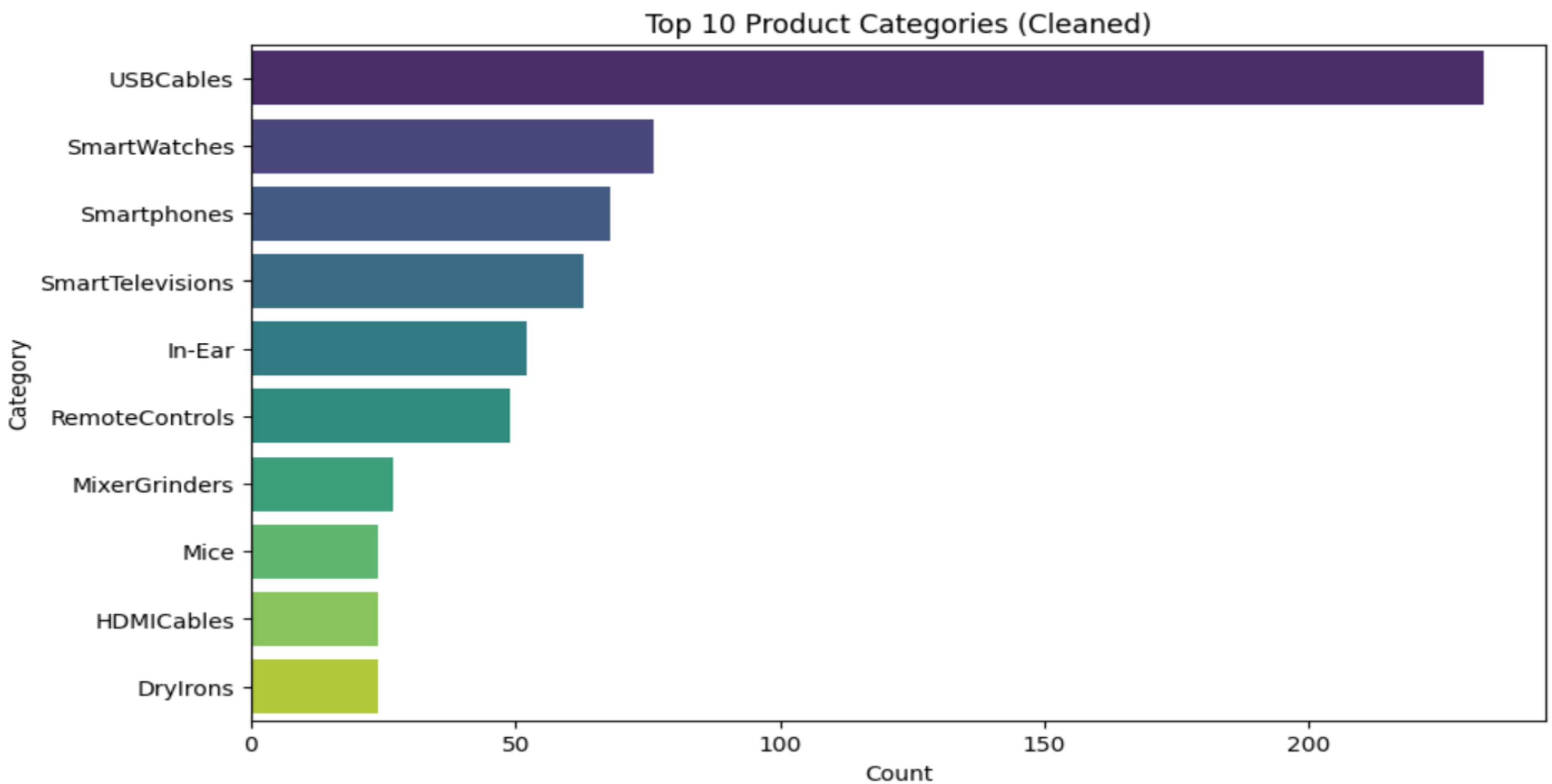
6-

```
top_categories = df['category'].value_counts().head(10)

plt.figure(figsize=(10,6))
sns.barplot(
    x=top_categories.values,
    y=top_categories.index,
    hue=top_categories.index,    # required for palette
    palette="viridis",
    legend=False                 # hide legend to keep chart clean
)

plt.title("Top 10 Product Categories")
plt.xlabel("Count")
plt.ylabel("Category")
plt.show()
```



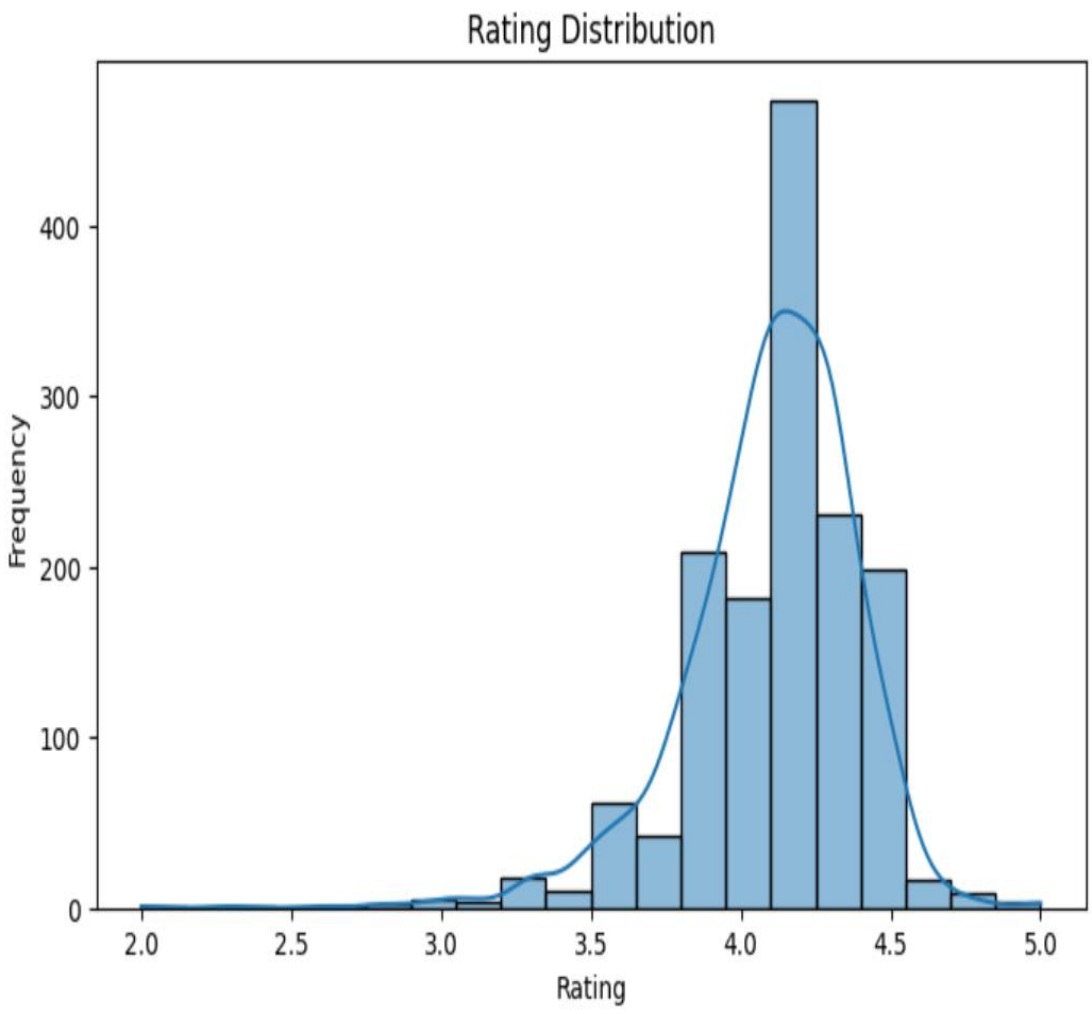
7-

```
]: top_cat = df['short_category'].value_counts().head(10)

plt.figure(figsize=(10,6))
sns.barplot(
    x=top_cat.values,
    y=top_cat.index,
    hue=top_cat.index,         # required when using palette
    palette="viridis",
    legend=False               # hides extra legend
)

plt.title("Top 10 Product Categories (Cleaned)")
plt.xlabel("Count")
plt.ylabel("Category")
plt.show()
```



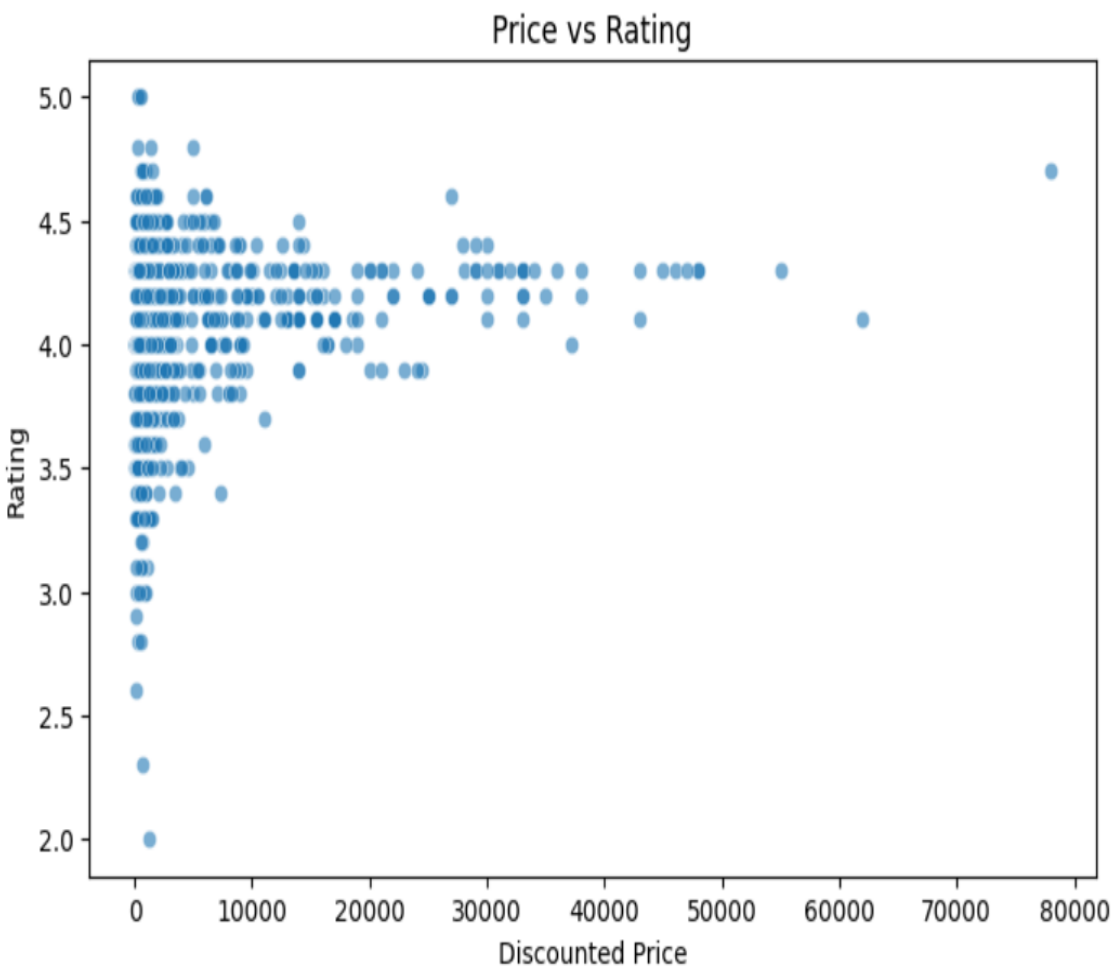
8-

```
plt.figure(figsize=(8,5))
sns.histplot(df['rating'], bins=20, kde=True)
plt.title("Rating Distribution")
plt.xlabel("Rating")
plt.ylabel("Frequency")
plt.show()
```



9-

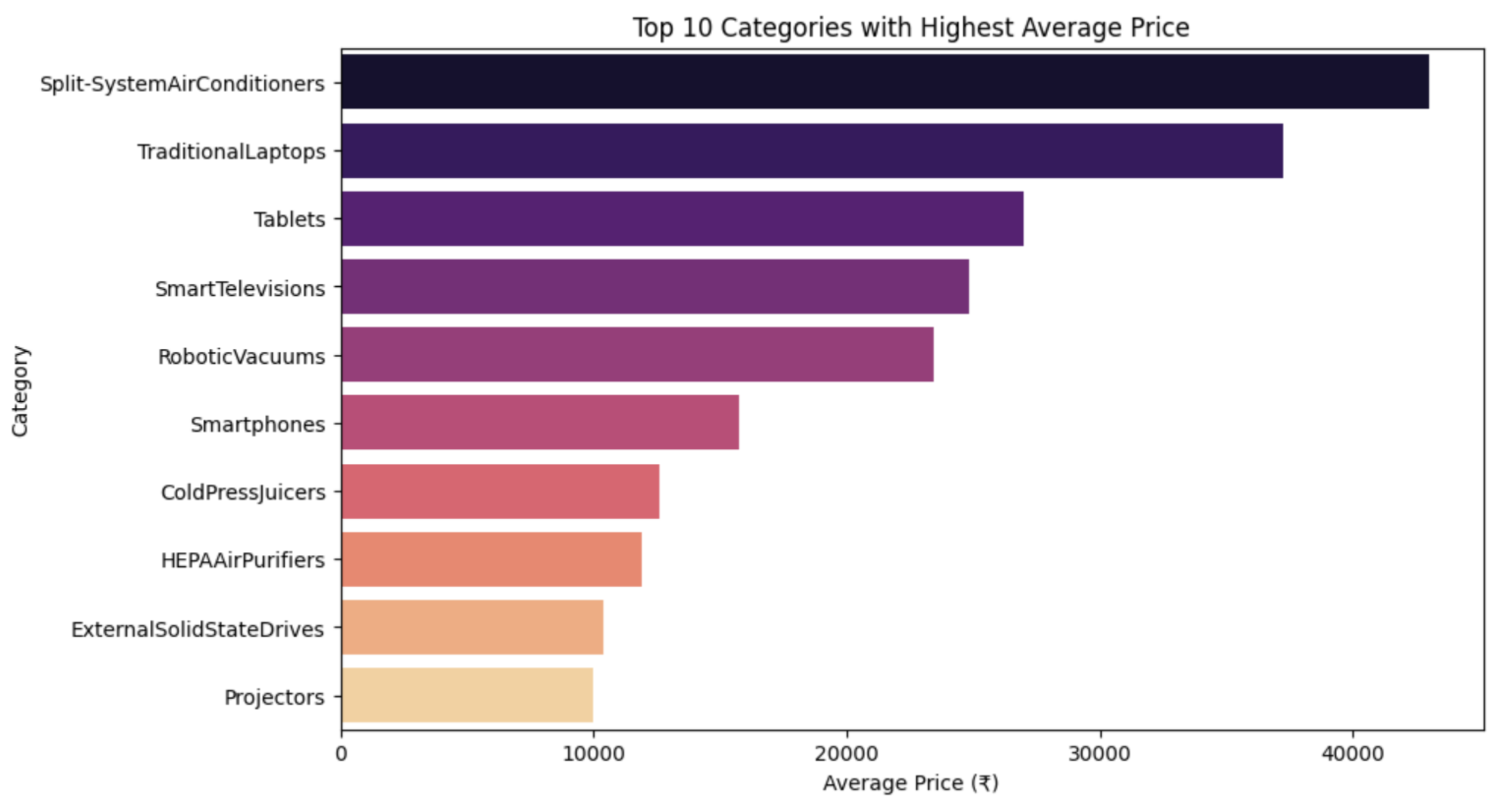
```
plt.figure(figsize=(8,5))
sns.scatterplot(x=df['discounted_price'], y=df['rating'], alpha=0.6)
plt.title("Price vs Rating")
plt.xlabel("Discounted Price")
plt.ylabel("Rating")
plt.show()
```



```
avg_price = df.groupby('short_category')['discounted_price'].mean().sort_values(ascending=False).head(10)
```

10-

```
plt.figure(figsize=(10,6))
sns.barplot(
    x=avg_price.values,
    y=avg_price.index,
    hue=avg_price.index,      # required when using palette
    palette="magma",
    legend=False              # hides unnecessary legend
)

plt.title("Top 10 Categories with Highest Average Price")
plt.xlabel("Average Price (₹)")
plt.ylabel("Category")
plt.show()
```

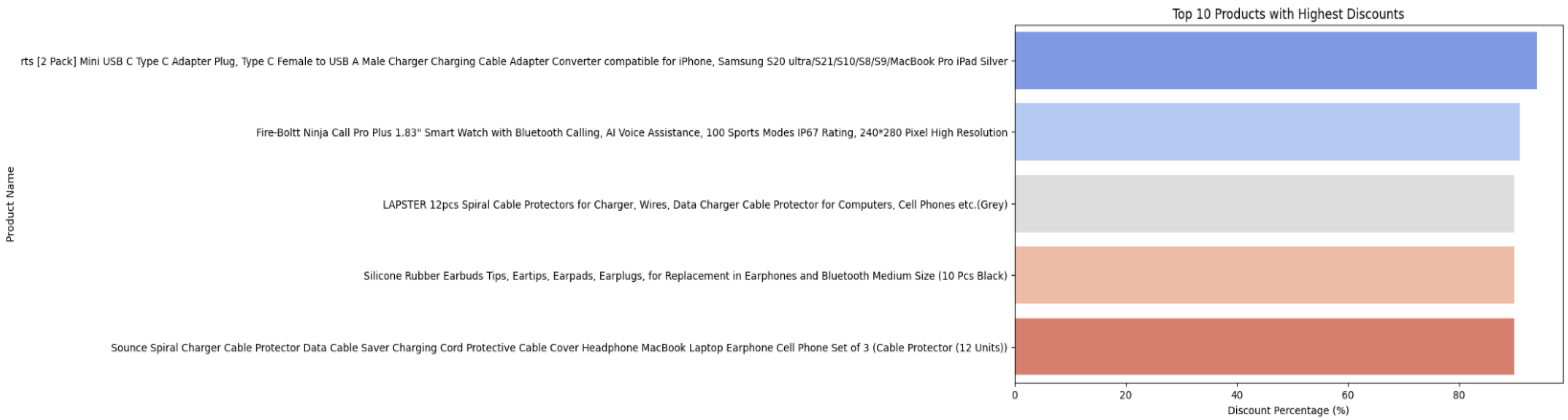


11-

```
top_discount = df.sort_values(by='discount_percentage', ascending=False).head(10)
top_discount[['product_name', 'short_category', 'actual_price', 'discounted_price', 'discount_percentage']]
```

[63]:

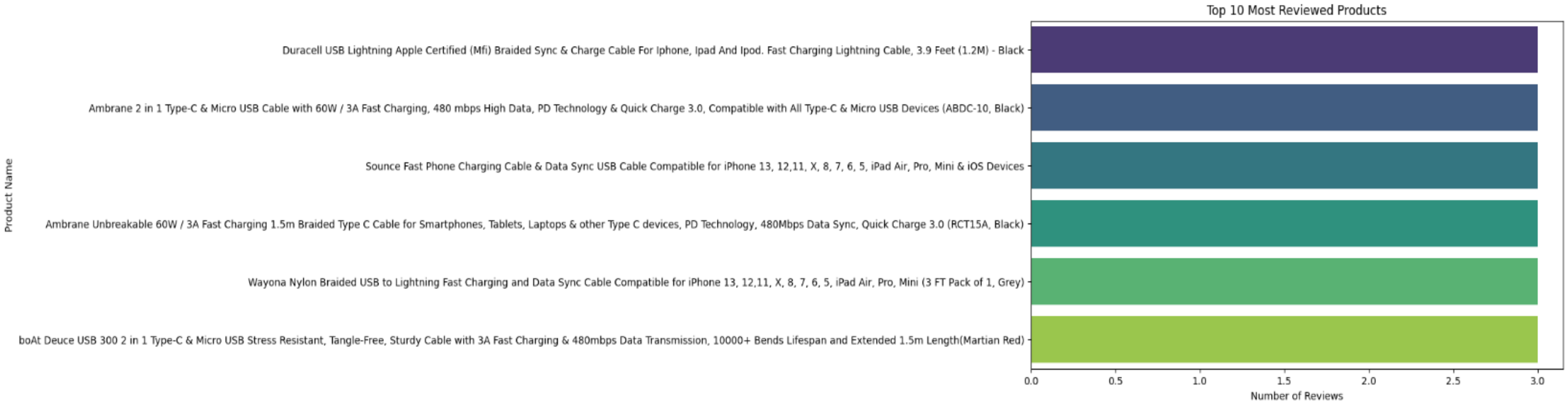
```
plt.figure(figsize=(10,6))
sns.barplot(
    x=top_discount['discount_percentage'],
    y=top_discount['product_name'],
    hue=top_discount['product_name'],   # required when using palette
    palette="coolwarm",
    legend=False                        # keep chart clean
)

plt.title("Top 10 Products with Highest Discounts")
plt.xlabel("Discount Percentage (%)")
plt.ylabel("Product Name")
plt.show()
```



12-

```
top_reviewed = df.sort_values(by='review_count', ascending=False).head(10)
top_reviewed[['product_name', 'short_category', 'rating', 'review_count']]
```

```
plt.figure(figsize=(10,6))
sns.barplot(
    x=top_reviewed['review_count'],
    y=top_reviewed['product_name'],
    hue=top_reviewed['product_name'],   # needed when using palette
    palette="viridis",
    legend=False                        # hides the extra legend
)

plt.title("Top 10 Most Reviewed Products")
plt.xlabel("Number of Reviews")
plt.ylabel("Product Name")
plt.show()
```

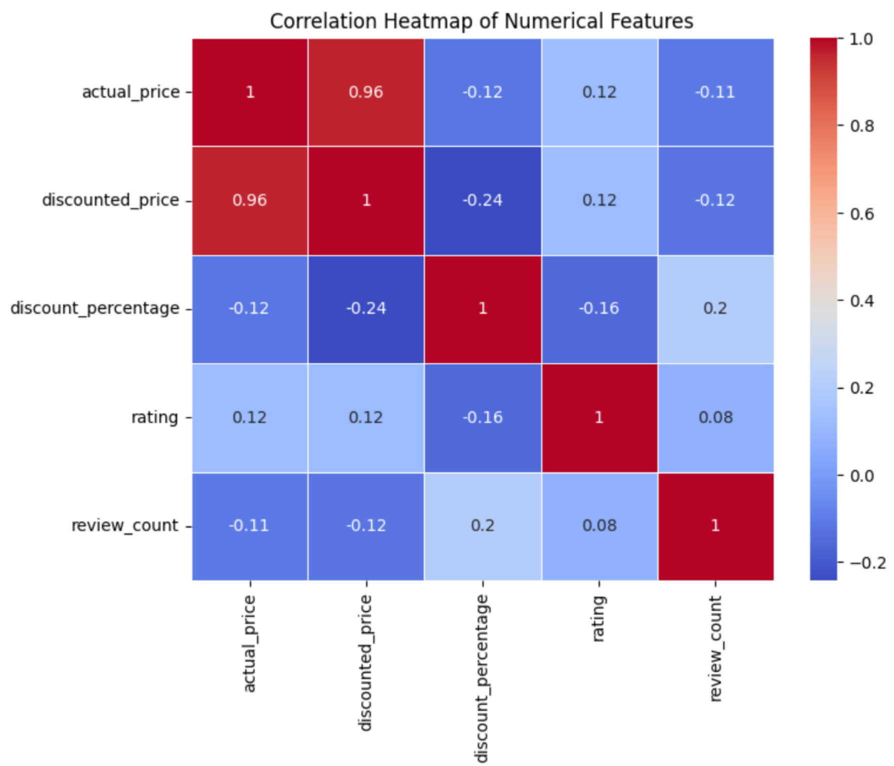


13-

```
numeric_df = df[['actual_price', 'discounted_price', 'discount_percentage', 'rating', 'review_count']]
```

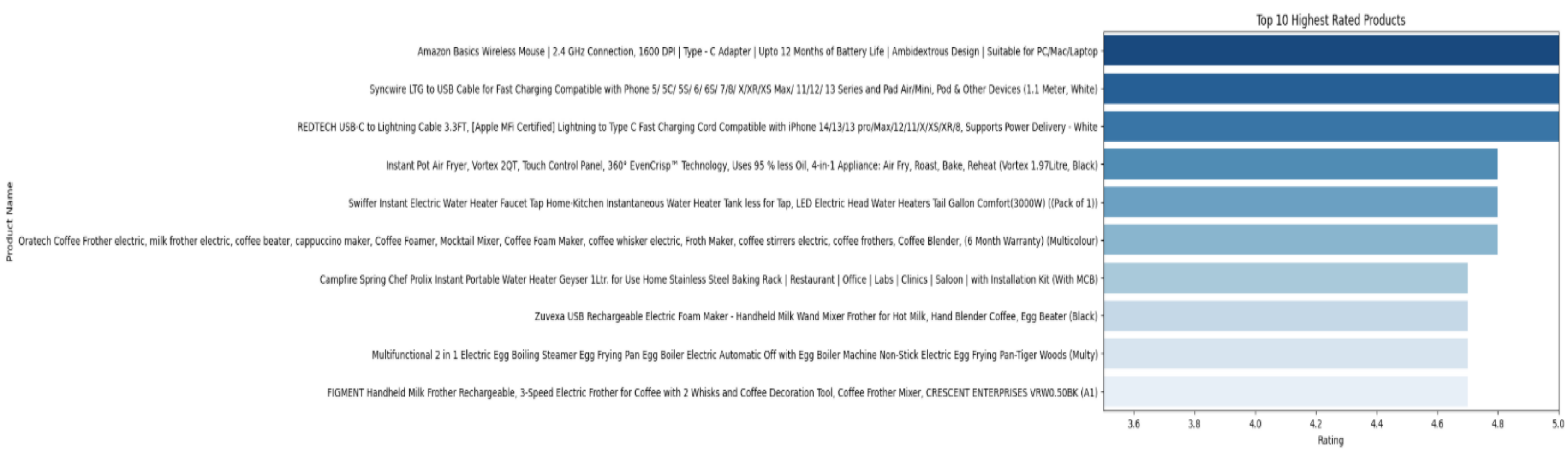
```
corr = numeric_df.corr()top_rated = df.sort_values(by='rating', ascending=False).head(10)
top_rated[['product_name', 'short_category', 'rating', 'discounted_price']]

corr
```

```
plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, cmap='coolwarm', linewidths=0.5)
plt.title("Correlation Heatmap of Numerical Features")
plt.show()
```



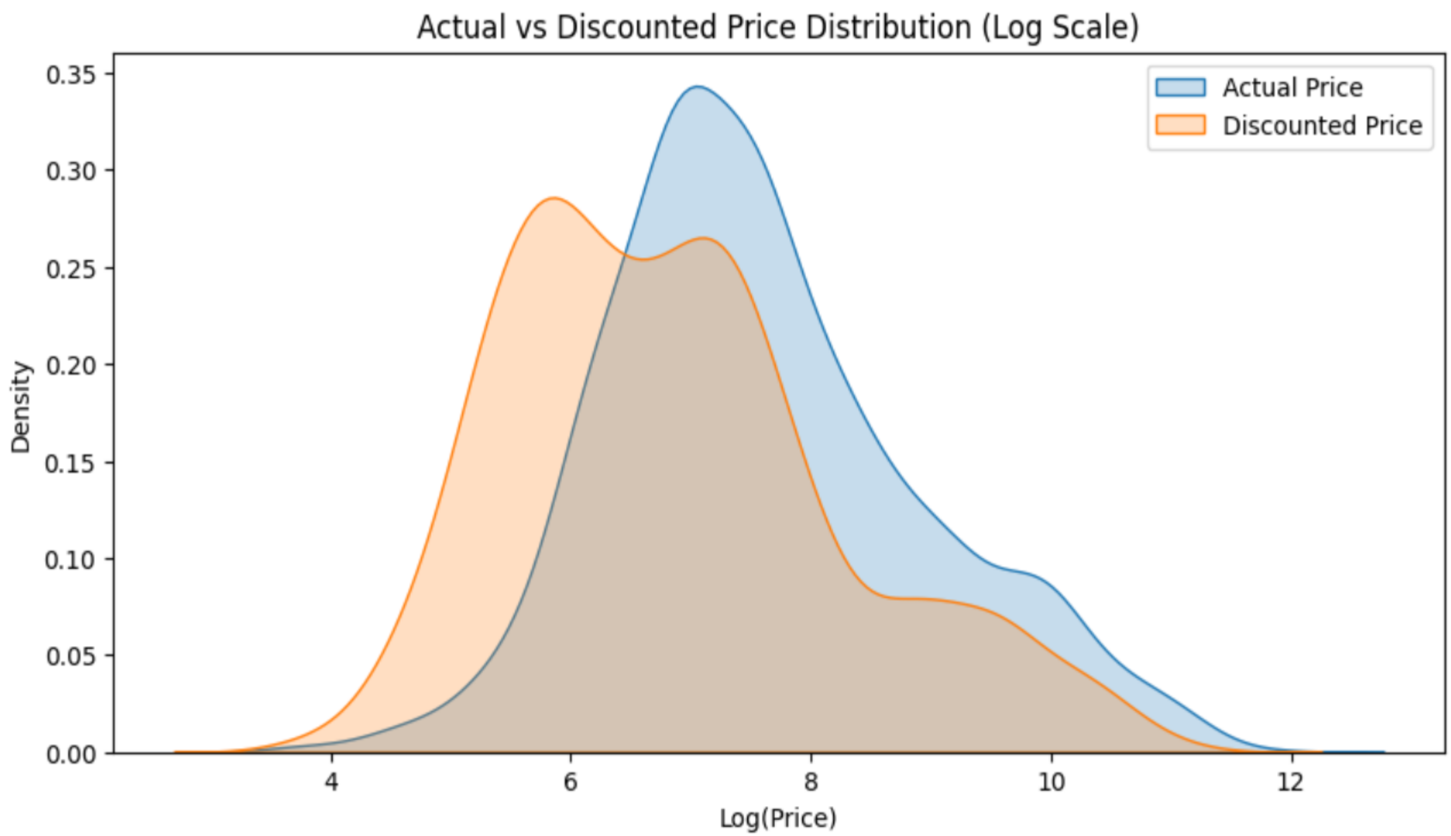
14-

```python
top_rated = df.sort_values(by='rating', ascending=False).head(10)
top_rated[['product_name', 'short_category', 'rating', 'discounted_price']]
plt.figure(figsize=(10,6))
sns.barplot(
    x=top_rated['rating'],
    y=top_rated['product_name'],
    hue=top_rated['product_name'],   # required when using palette
    palette="Blues_r",
    legend=False                     # hides unnecessary legend
)

plt.title("Top 10 Highest Rated Products")
plt.xlabel("Rating")
plt.ylabel("Product Name")
plt.xlim(3.5, 5)
plt.show()
```



15-

```
plt.figure(figsize=(10,5))

sns.kdeplot(np.log1p(df['actual_price']), label='Actual Price', fill=True)
sns.kdeplot(np.log1p(df['discounted_price']), label='Discounted Price', fill=True)

plt.title('Actual vs Discounted Price Distribution (Log Scale)')
plt.xlabel('Log(Price)')
plt.ylabel('Density')
plt.legend()
plt.show()
```



Key Insights

1. Electronics dominate listings
2. Price distribution is skewed
3. Accessories receive high discounts
4. Ratings mostly 3.8–4.4
5. Price does not affect rating
6. Most reviews are for low-cost items

Business Recommendations

- Promote high-rated low-visibility products
- Bundle accessories
- Improve premium product quality
- Optimize discount strategy
- Improve descriptions of low-rated categories

Conclusion

Analysis reveals customer behavior, pricing impact, and category performance. Useful for pricing, marketing, and inventory strategy.